CASE-STUDY PRACTICE | CAPSTONE 10 | STUDY EDITION

LLM Inference Serving at Scale

Scenario questions on serving a 13B-parameter model to a thousand concurrent conversations, each opening into deeper follow-ups, with answers that explain how to think. Built on the AI System Design Primer case study of the same name.

Jump to the contents | The case in one page | Numbers to know | Decision table | Sources

HOW TO STUDY WITH THIS BOOK

1. Read the amber **concepts box** at the start of a part and look up anything unfamiliar. 2. Read a **scenario** and, before reading further, write down what you would ask and what you would check. 3. Read the green **how to think** box, which names the mental move the question tests. 4. Walk down the nested steps: cover each answer, predict it, then compare. Each answer exposes the next question. The last step of every question asks for the general lesson in one sentence.

The pattern behind most questions

- A number or a technique arrives with authority: a published benchmark, a cost estimate, a shard count, a rate.
- The interviewer asks what it assumes, and what happens when the assumption fails.
- A strong answer names the hidden assumption, puts a number on it, and says how to measure it on your own data.
- The last step usually reframes the original question.

EVIDENCE TAGS

source-confirmed: checked against the named primary source. **illustrative**: this document's planning assumption or round figure. **recomputed**: arithmetic re-derived from the stated inputs. **docs, 2026-10-17**: read from the vLLM documentation pages on that date. **reasoned**: an argument from the design, not tested.

A NOTE ON SOURCES AND ON WHAT WAS CHECKED

The scenarios are written for this document; none comes from a real company question bank. Nothing here was run on a GPU, a model server or a load test: figures are the paper's, the blog post's, the vLLM documentation's, or arithmetic from stated inputs, and hardware speeds are round illustrative numbers. The last page lists the sources and says how much of each was read. A separate Mock Interview edition (answers hidden) uses the same questions and numbering.

Contents

Every line below is a link: click it to jump to that page. The outline panel of your PDF viewer lists the same entries, and the “Contents” link in every page footer brings you back here.

Front matter

A The case in one page 3

B Numbers to know 4

C Decision table 5

Parts at a glance

1 Framing the problem: targets, memory and cost 6

2 PagedAttention: from operating-system paging to KV blocks 12

3 Scheduling, caching and routing 17

4 Operating at scale: dashboards, availability, growth and safety 22

Sources and confidence 27

Every question, by part

Part 1 Framing the problem: targets, memory and cost 6

01 Write the targets before the architecture Framing 6

02 Review the fleet-size estimate Sizing 8

03 Memory or compute: what limits the batch? Mechanism 9

04 Price a million tokens Cost 10

Part 2 PagedAttention: from operating-system paging to KV blocks 12

05 Why contiguous allocation wastes 60 to 80% Mechanism 12

06 Find the bug in the block allocator Code review 13

07 Choose the block size Design choice 15

08 Twenty users share a system prompt Sharing 16

Part 3 Scheduling, caching and routing 17

09 The GPU is out of blocks in the middle of decoding Operations 17

10 One long prompt stalls every stream Debugging 18

11 Keep the conversation's cache, or recompute it? Cache design 19

12 The load balancer is round-robin Routing 20

Part 4 Operating at scale: dashboards, availability, growth and safety 22

13 Three incidents, one dashboard Monitoring 22

14 What availability can we promise? Availability 23

15 Traffic grows tenfold Scale 24

16 Two tenants share a prefix cache Security 25

17 A blog post says 33 times cheaper Evidence 26

The case in one page

A summary of the case study to read before the questions.

Problem	Serve one already-trained 13B-parameter model to 1,000 concurrent conversations on 80 GB GPUs within latency targets and a cost target. GPU memory runs out long before compute does.
Core idea	Store each request's KV cache in small fixed-size blocks, scattered anywhere in GPU memory and tracked by a per-request block table, as an operating system pages virtual memory. Admit requests when blocks are free.
Targets	p50 and p99 time to first token under 200 ms and 1.5 s. p50 and p99 inter-token latency under 40 ms and 120 ms. Cost per million tokens against a stated budget. All illustrative.
Headline results	Earlier systems held real token states in only 20.4% to 38.2% of KV-cache memory. vLLM held them in 96.3% and raised throughput 2 to 4 times at the same latency (source-confirmed, the paper).
KV cache per token	2 × 40 layers × 5,120 hidden × 2 bytes = 819,200 bytes, 800 KB, for OPT-13B (source-confirmed). A 2,500-token request holds about 2.0 GB.
Fleet size	26 full-length requests per 80 GB GPU with perfect packing, 25 with block allocation: about 40 GPUs for 1,000 conversations. Contiguous allocation needs about 112 to 200.
Cost	About $1.11 per million output tokens at $2.50 per GPU-hour and full utilization, three to five times that without block allocation (illustrative).
Easy-to-miss traps	A 13B model sized with 7B dimensions. A sizing that assumes zero waste. A 200 ms first-token target that a cold 2,000-token prompt cannot meet. Round-robin routing that discards cached prefixes. A prefix cache shared across tenants.

Numbers to know

Each figure carries the tag that says how it was obtained. Hardware speeds and the GPU price are round illustrative numbers, not measurements.

Quantity	Value	Evidence
KV cache per token (OPT-13B)	2 × 40 × 5,120 × 2 = 819,200 B = 800 KB; a 2,048-token request up to 1.6 GB	source-confirmed (the paper)
One full-length request	2,500 tokens × 0.8 MB = 2.0 GB	recomputed
KV-cache budget per GPU	80 GB − 26 GB weights − 2 GB overhead = 52 GB	illustrative (80 GB GPU, 2 GB overhead)
Requests and GPUs, perfect packing	52 ÷ 2.0 = 26 per GPU; 1,000 ÷ 26 = 38.5, so 39 GPUs	recomputed
Share of KV memory holding real tokens	Earlier systems 20.4% to 38.2%; vLLM 96.3%	source-confirmed (the paper, Figure 2)
Fleet with waste	Earlier systems 5 to 9 requests per GPU, about 112 to 200 GPUs; vLLM 25 per GPU, 40 GPUs	recomputed
Throughput target	1,000 streams × 25 tokens/s (40 ms a token) = 25,000 tokens/s = 90 million tokens/hour	illustrative
Cost per million tokens	$2.50 per GPU-hour: 40 GPUs = $100/h = $1.11 per million; 112 to 200 GPUs = $3.11 to $5.56	illustrative price; recomputed
One decode step	(26 GB weights + 45 GB KV) ÷ 2,000 GB/s = about 35 ms	illustrative bandwidth; recomputed
Prompt processing	2 × 13e9 × 2,000 tokens = 5.2e13 operations: 0.17 s at 300 trillion/s, 0.4 s at 40% of peak	illustrative speed; recomputed
Block size and waste	16 tokens by default; at most 15 unused tokens per request, about 0.3% of the cache at 25 requests	source-confirmed (default); recomputed
Swap versus recompute, 2,000 tokens	1.6 GB at 25 GB/s = 64 ms each way; recompute 0.17 to 0.43 s	illustrative bandwidth; recomputed
Availability	0.9995 × 0.999 × 0.9999 = 99.84% (14 h/year); with a backend pair 99.94% (5.3 h/year)	recomputed (Lesson 3 values)
Ten times the load	10,000 conversations: 400 GPUs; 8-bit weights 313; 8-bit weights and cache 154	recomputed
Character.AI	At least 33 times lower serving cost since 2022, against its own earlier cost; 13.5 times below a commercial-API competitor; about 20,000 queries/s	source-confirmed (its blog post)
vLLM scheduling policy	First-come-first-served; latest-arrived requests are preempted first; swap or recompute	source-confirmed (the paper)

Decision table

What this design chooses, and when to choose otherwise.

Decision	Default here	Choose otherwise when	Evidence
KV-cache allocation	Paged blocks	Never for a shared multi-request server; contiguous only for a single-request tool	source-confirmed
Block size	16 tokens	Larger if kernels are the bottleneck and requests are long; smaller if requests are short	source-confirmed; reasoned
Admission	Free blocks, not a request count	A hard request cap only as an extra safety limit	source-confirmed
Preemption mode	Recompute in current vLLM	Swap when host-to-GPU bandwidth is high and recompute is long; test both	docs, 2026-10-17; reasoned
Prefix reuse	On, with cache isolation per tenant	Off for single-use prompts or where side channels matter more than speed	docs, 2026-10-17; reasoned
Headroom	Run below 100% KV utilization	Close to full only when cost matters more than p99 and demand is stable	reasoned
Model precision	16-bit	8-bit or lower when memory is the limit and an evaluation shows acceptable quality	reasoned
Routing	Session affinity with consistent hashing	Round-robin only if prefix reuse is not part of the latency plan	reasoned

PART 1

Framing the problem: targets, memory and cost

Turning a vague request into numbers, then checking the numbers against what the hardware can do.

READ THESE FIRST: CONCEPTS COVERED IN THIS PART

Time to first token (TTFT)	The wait between sending a request and receiving the first generated token. It includes any queueing and the processing of the whole prompt (prefill), so it grows with prompt length.
Inter-token latency	The gap between one generated token and the next once streaming has started. It is set by the time of one decode step, so it grows as more requests share the batch.
p50 and p99	Percentile latencies. p50 is the median: half of requests are faster. p99 is the value that 99% of requests beat, so it describes the slow tail that users complain about.
SLA and SLO	A service-level agreement (SLA) is a committed target, often with penalties. A service-level objective (SLO) is the internal target a team holds itself to, usually stricter, so that the SLA is met with room to spare.
Prefill and decode	Prefill processes all prompt tokens at once and is limited by compute. Decode produces one token per step per request and is limited by how fast weights and cached keys and values can be read from GPU memory.
KV cache	The stored key and value vectors of every token already processed, for every layer, kept so that each new token does not recompute them. Bytes per token = 2 (key and value) × layers × hidden size × bytes per number.
Continuous batching	Requests join and leave the running batch at every step, instead of waiting for a whole batch to finish. It keeps the GPU busy, and it is why memory for many partly finished requests must be managed.
Quantization	Storing numbers with fewer bits (for example 8-bit integers instead of 16-bit floats) to shrink weights or the KV cache, at some cost in output quality that has to be measured.
Where to read	Kwon et al.: PagedAttention (vLLM paper) | vLLM documentation: optimization and tuning

01 Write the targets before the architecture

FRAMING

SCENARIO

A product manager asks for chat serving for a 13B-parameter model that is “fast and cheap.” Engineers are already arguing about the batch size. About 1,000 conversations will be active at once, with 2,000 tokens of context on average and 500 generated tokens each.

HOW TO THINK ABOUT IT

Adjectives are not requirements. Turn each one into a number with a percentile, say which target wins when two conflict, and then check that the numbers are possible on the hardware before anyone designs against them.

Q What targets do you write down?

A Four, each as a number. A first-token wait, a streaming speed, a system-wide rate and a price.

- **TTFT:** p50 under 200 ms and p99 under 1.5 s (illustrative).
- **Inter-token latency:** p50 under 40 ms and p99 under 120 ms per token (illustrative). 40 ms is 25 tokens a second per stream.
- **Throughput:** 1,000 streams × 25 tokens a second = 25,000 tokens a second at peak.
- **Cost:** dollars per million tokens, GPU-hour price divided by tokens served.
- Keep an SLO a little stricter than the SLA you promise outside, so that normal variation does not breach the promise.

Q Which two targets pull against each other?

A Throughput and latency, through the batch size. A bigger batch shares each read of the model weights across more requests, which raises tokens per second for the whole system, and makes every request's step longer.

- Lesson 3's illustrative table: batch 1 gives 20 tokens a second at 50 ms a token, batch 32 gives 400 tokens a second at 80 ms. Throughput rises 20 times while each stream slows by 60%.
- Write the priority down before load arrives, for example “keep p99 inter-token latency under 120 ms first, then maximize throughput.” Without it, someone lowers the batch size during an incident and quietly breaks the throughput target.

Q Can a 200 ms p50 TTFT be met for a 2,000-token prompt on one GPU?

A Check it with arithmetic. Processing a prompt costs about 2 × parameters × tokens operations, ignoring attention's extra term.

- 2 × 13 billion × 2,000 = 5.2 × 10^13 operations. At an assumed 300 trillion operations a second that is about **0.17 s** at peak, and about **0.4 s** at a realistic 40% of peak, before any queueing.
- So a cold 2,000-token prompt cannot meet a 200 ms median. The target is only reachable when most of the prompt is already cached: 100 new tokens on top of 1,900 cached cost 2 × 13e9 × 100 = 2.6 × 10^12 operations, about 9 ms.

Q How do you write the TTFT target so it can be met honestly?

A Split it by whether the prompt prefix is cached.

- For example: p50 under 200 ms for turns whose context is mostly cached, and a separate, looser target of around 0.5 to 1 s for cold prompts (illustrative).
- Then track the cache hit rate as its own metric, since the first target holds only while the hit rate does (Question 11).

Q Reframe it.

A A target is a claim about the hardware. Before committing to one, do the one-line arithmetic that says whether any design could meet it.

Basis: Target values and the 2,000-token workload are illustrative; the batch-size table is Lesson 3's illustrative example; prefill arithmetic uses the 2 × parameters × tokens rule with an assumed 300 trillion operations a second (recomputed).

02 Review the fleet-size estimate

SIZING

SCENARIO

A colleague sizes the fleet for 1,000 concurrent conversations on 80 GB GPUs serving a 13B-parameter model in 16-bit precision. Their notes are below, and they conclude that 24 GPUs are enough. Find what is wrong.

```
# the colleague's estimate
kv_per_token = 2 * 32 * 4096 * 2          # layers, hidden size, bytes -> 512 KB
request_gb   = 1.22                       # 2,500 tokens
per_gpu      = 52 / 1.22                  # 42 requests per GPU
gpus         = 1000 / 42                  # 24 GPUs
```

HOW TO THINK ABOUT IT

Check every input against the model you actually serve, then check what each division assumes. A tidy chain of arithmetic is only as good as its first number.

Q What is wrong with the per-token figure?

A The shape. 32 layers and a hidden size of 4,096 is the shape of a 7B model, not a 13B one. The paper's 13B model (OPT-13B) has 40 layers and a hidden size of 5,120.

- Bytes per token = 2 × 40 × 5,120 × 2 = 819,200, which is **800 KB**. The paper gives the same number and notes that a 2,048-token request can take up to 1.6 GB (source-confirmed).
- A quick consistency check catches it: the notes use 26 GB of weights, which is a 13B model, with the KV figure of a 7B model, whose weights would be 14 GB.

Q Redo the chain with the right figure.

A A full request is 2,500 tokens × 0.8 MB = 2.0 GB. The cache budget is 80 − 26 − 2 = 52 GB, so 52 ÷ 2.0 = **26 requests per GPU**, and 1,000 ÷ 26 = 38.5, so **39 GPUs**.

- The estimate of 24 was about 38% too low (15 GPUs short of 39).
- Sizes are rounded to decimal units here. Exact binary units move the answer by under 10%.

Q What else does the estimate assume?

A That every byte of the 52 GB holds real token states. That is perfect packing, which only block-level allocation approaches.

- The paper measured the share of KV memory holding real tokens: 20.4% to 38.2% in earlier systems and 96.3% in vLLM (source-confirmed).
- Applied to this fleet: 26 × 0.382 ≈ 9 and 26 × 0.204 ≈ 5 requests per GPU for earlier systems, so about **112 to 200 GPUs**. For vLLM, 26 × 0.963 ≈ 25 per GPU, so **40 GPUs**.

Q Is sizing every request at its peak length right?

A It is conservative, and that is usually correct for memory. A request reaches 2,500 tokens only at its very end, so averaged over its life it holds about 2,250 tokens, about 10% less.

- But memory must cover the moment when many requests are long at once, so size for a high percentile of the real length distribution, not the average.
- Take the length distribution from logs. A few very long conversations can matter more than the mean.

Q What do you verify before ordering hardware?

A The model's real configuration, the real length distribution and the real overhead.

- Read the layers, hidden size and the number of key-value heads from the model's config. A model with grouped-query attention has fewer key-value heads, and its KV cache per token is correspondingly smaller: use kv_heads × head_dim in place of the hidden size.
- Measure the activation and runtime overhead instead of assuming 2 GB.

Q Reframe it.

A Check the inputs of an estimate before its arithmetic, then ask what the division assumes. A wrong input and an unstated assumption are the two usual ways a clean estimate fails.

REFERENCE CODE

```
kv_per_token = 2 * 40 * 5120 * 2          # the 13B model's real shape -> 800 KB
request_gb   = 2500 * 0.8 / 1000          # 2.0 GB
per_gpu      = 52 / 2.0                   # 26 requests, if packing is perfect
gpus         = 1000 / 26                  # 38.5, so 39 GPUs
# with block-level allocation, 96.3% of memory holds real tokens: 25 per GPU, 40 GPUs
```

Basis: Model shape and 800 KB per token are from the paper (source-confirmed). The 80 GB GPU, 2 GB overhead and 1,000 conversations are illustrative. Everything else is arithmetic (recomputed).

03 Memory or compute: what limits the batch?

MECHANISM

SCENARIO

The team notices that adding requests to the batch raises throughput until it stops helping. Someone proposes buying GPUs with more compute. Another says memory is the problem. You have the sizes from Question 2 and no benchmarks.

HOW TO THINK ABOUT IT

Count what one decode step has to move and compute, then see which resource is closer to its limit. Use round hardware numbers and say they are assumptions.

Q What does one decode step have to read?

A The model weights once, and the cached keys and values of every request in the batch.

- Weights: 26 GB for a 13B model in 16-bit precision.
- KV cache: 25 running requests × about 1.8 GB each (2,250 tokens × 0.8 MB at average length) = about 45 GB.

Q How long does that take?

A With an assumed 2 TB a second of memory bandwidth: (26 + 45) GB ÷ 2,000 GB/s = about **35 ms** a step.

- That is close to the 40 ms inter-token target, so the targets and the sizing agree with each other.
- It ignores kernel inefficiency and communication, so real steps run slower. Treat it as a floor.

Q Why does batching raise throughput at all?

A The 26 GB of weights are read once per step no matter how many requests share it.

- One request: (26 + 1.8) ÷ 2,000 = 13.9 ms a step, so 72 tokens a second.
- Twenty-five requests: 35.5 ms a step, and 25 ÷ 0.0355 = 704 tokens a second. Throughput rises about 10 times while the step grows only about 2.6 times.

Q What stops you adding more requests?

A Two things, both about memory. KV capacity: the 52 GB budget holds at most 26 full-length requests. KV bandwidth: each added request adds to the bytes read every step, so every stream gets slower.

- Compute is nowhere near its limit. 25 requests × 2 × 13 billion = 6.5 × 10^11 operations a step, about 2 ms at 300 trillion a second, around 6% of the 35 ms step.

Q What would you buy, and what would you change?

A More memory capacity and bandwidth per dollar, and smaller KV caches. Not more compute.

- Levers on the KV size: quantize the cache, or choose a model with grouped-query attention at model-selection time. It is a training-time property and cannot be retrofitted.
- Levers on waste: block-level allocation, so the memory you have holds real tokens.

Q Reframe it.

A Find the resource that is closest to its limit by counting bytes and operations for one step. The answer here is memory, and it decides what is worth buying.

Basis: Bandwidth (2 TB a second) and compute (300 trillion operations a second) are round illustrative figures; sizes from Question 2; the rest is arithmetic (recomputed). Real systems run below both peaks.

04 Price a million tokens

COST

SCENARIO

Finance asks for the cost of a million generated tokens and whether the memory-management technique matters to the bill. Use a GPU price of $2.50 an hour (illustrative) and the sizing from Question 2.

HOW TO THINK ABOUT IT

Cost per token is the cost of the fleet divided by the tokens it serves. Compute it once, then list every assumption that makes the real figure worse.

Q What is the cost per million tokens for the block-allocated fleet?

A 40 GPUs × $2.50 = $100 an hour. The fleet serves 1,000 streams × 25 tokens a second = 25,000 tokens a second = 90 million tokens an hour. So $100 ÷ 90 = about **$1.11 per million tokens**.

Q What does the memory technique change?

A Without it the same traffic needs about 112 to 200 GPUs: $280 to $500 an hour, so **$3.11 to $5.56 per million tokens**, three to five times as much.

- Memory waste is a cost line, not only a performance detail.

Q What does the calculation assume that real traffic will not satisfy?

A Full utilization, continuous streaming, output tokens only and peak-length sizing.

- At 50% average utilization the cost doubles. Traffic has a daily shape, so idle capacity is the usual reason real cost exceeds the estimate.
- Prompt processing is left out, so a workload of long prompts and short answers costs more per output token than shown.

Q Where is the next saving?

A Utilization first, then memory levers.

- Scale the fleet with the daily load where the cluster manager and the warm-up time allow it.
- 8-bit weights free 13 GB, so 65 ÷ 2.0 = 32 requests per GPU and 1,000 ÷ 32 = 32 GPUs: $80 an hour, about **$0.89 per million tokens**, if an evaluation shows acceptable quality.
- Share prefixes across requests (Question 8).

Q How do you track it in production?

A GPU-hours billed divided by tokens served, per model and per tenant, next to utilization.

- A rising cost with a flat fleet means fewer tokens are being served: +30% cost per token is 1 ÷ 1.3 = 77% of the tokens, a 23% drop (Question 13).

> **Q Reframe it.**
> **A** Cost per token is capacity divided by utilization. The memory waste factor hides inside capacity, which is why a memory technique shows up on a finance report.

Basis: GPU price is illustrative; sizing from the paper's waste shares (source-confirmed) and Question 2; all dollar figures are arithmetic (recomputed).

PART 2

PagedAttention: from operating-system paging to KV blocks

Why contiguous allocation wastes most of the cache, how blocks and block tables fix it, and what sharing blocks makes possible.

READ THESE FIRST: CONCEPTS COVERED IN THIS PART

Reservation, internal and external fragmentation	Three kinds of wasted KV memory in a contiguous scheme. Reservation: space held for tokens not yet generated. Internal fragmentation: unused space inside a request's block. External fragmentation: gaps between blocks too small or awkward to reuse.
Virtual memory and paging	An operating system splits a program's logical address space into fixed-size pages and maps each to any free physical frame through a page table. The program need not know where its pages physically sit.
KV block and block table	PagedAttention splits each request's KV cache into fixed-size blocks (16 tokens by default in vLLM) kept anywhere in GPU memory. A per-request block table records which physical block holds which positions.
Copy-on-write and reference count	Several owners share one physical copy and a counter records how many. The first owner that needs to modify the data gets a private copy, and the count of the shared one drops.
Parallel sampling and beam search	Decoding methods that produce several output sequences from one prompt. Their sequences share the prompt's KV cache, so block sharing saves memory.
Prefix caching	Keeping the blocks of a prompt prefix after a request ends so a later request with the identical prefix reuses them instead of recomputing. A block is identified by its own tokens plus the tokens before it.
Block size	The number of tokens in one KV block. Small blocks waste less space per request; large blocks make each kernel call more efficient. The unused space in a request's last block is its internal fragmentation.
Where to read	Kwon et al.: PagedAttention (vLLM paper) | vLLM documentation: automatic prefix caching

05 Why contiguous allocation wastes 60 to 80%

MECHANISM

SCENARIO

A teammate argues: “Memory is cheap. Give each request one contiguous region sized for the maximum length, 2,048 tokens, and keep the code simple.” You serve the 13B model, 0.8 MB of KV cache per token, and a typical conversation ends at about 400 tokens.

HOW TO THINK ABOUT IT

Follow the memory of a single request through its life and name each kind of waste where it appears. Then look for the measurement that decides whether the waste matters in practice.

Q Where does the waste come from?

A Three places. Reservation, internal fragmentation and external fragmentation.

- **Reservation and internal:** the request reserves 2,048 × 0.8 MB = 1.6 GB and uses 400 × 0.8 MB = 0.32 GB. That is 1.28 GB, 80% of its region, idle, and nobody else can use it while the request runs.
- **External:** requests of different lengths finish at different times, leaving gaps between regions that are too small for the next request even when the total free memory is enough.

Q What did the paper measure?

A Only 20.4% to 38.2% of KV-cache memory held real token states in the existing systems it profiled, and 96.3% in vLLM (source-confirmed, Figure 2).

- That is the origin of the “60 to 80% wasted” figure for naive allocation.

Q Why not just compact memory now and then?

A Compaction means copying very large tensors while requests are being served.

- One request's cache is about 1.6 GB. Moving many of them stalls the streams that depend on them, and compaction has to be repeated as gaps reappear. It treats a symptom.

Q What does paging fix, and what does it leave?

A It fixes reservation and external fragmentation: blocks are taken one at a time as a sequence grows, and any free block fits any request, so no gap is ever too small.

- It leaves internal fragmentation in each request's last block: at most 15 tokens with 16-token blocks, about 12 MB, against 1.28 GB wasted in the contiguous scheme.

Q Reframe it.

A Memory management is a question of how much of what you reserved holds data. Measure that share first, then choose the mechanism.

Basis: Waste shares are from the paper's Figure 2 (source-confirmed); the 2,048-token reservation and 400-token conversation are illustrative; the rest is arithmetic (recomputed).

06 Find the bug in the block allocator

CODE REVIEW

SCENARIO

A new engineer has written the allocator below. It passes a simple test where requests start with an empty prompt. In staging, long prompts produce garbage output, and sometimes the process crashes under load. Block size is 16 tokens.

```
BLOCK_SIZE = 16

class Allocator:
    def __init__(self, total_blocks):
        self.free = list(range(total_blocks))
        self.tables = {}

    def on_decode_step(self, request_id, generated):
        # called once per decode step with the number of tokens generated so far
        table = self.tables.setdefault(request_id, [])
        if generated > len(table) * BLOCK_SIZE:
            table.append(self.free.pop())
```

HOW TO THINK ABOUT IT

Trace the code by hand with a concrete request at the block boundaries, then ask what the function does when the pool is empty. Boundary values find most allocator bugs.

Q What does it do wrong for a request with a 40-token prompt?

A It counts only generated tokens, so the prompt's blocks are never allocated.

- The prompt needs ceil(40 ÷ 16) = 3 blocks before decoding starts. The code allocates its first block only at generated = 1, and that block counts toward generated tokens.
- The request then runs with 3 blocks fewer than it needs: its cache is written into memory it does not own, or read from it. The symptom is garbage output, and an empty-prompt test cannot show it.

Q What goes wrong when memory is full?

A free.pop() on an empty list raises an exception and crashes the step. A full pool is a normal event, and the allocator should report it.

- Return a failure value so the scheduler can preempt a request and retry (Question 9).

Q State the corrected rule.

A Blocks needed = ceil(total tokens ÷ 16), where total tokens means prompt plus generated, counting the token about to be written.

- Boundary check: 41 tokens need 3 blocks, 48 still fit in 3, and 49 needs a 4th.
- Allocate in a loop until the table holds enough blocks, so a long prompt arriving at once is handled the same way as one token at a time.

Q What would you test?

A Invariants, at every step.

- Blocks held = ceil(total tokens ÷ 16) for every running request.
- Free blocks + held blocks = total blocks, so nothing leaks, including after cancellation.
- Freeing twice is harmless, and tokens 16, 17, 48 and 49 behave as above.

Q Reframe it.

A Test code at its boundaries and at its failure path. The ordinary case rarely hides the bug.

REFERENCE CODE

```
def blocks_needed(total_tokens):
    return -(-total_tokens // BLOCK_SIZE)            # ceiling division

class Allocator:
    def ensure_capacity(self, request_id, total_tokens):
        # total_tokens = prompt + generated, counting the token about to be written
        table = self.tables.setdefault(request_id, [])
        while len(table) < blocks_needed(total_tokens):
            if not self.free:
                return False                          # caller preempts someone, then retries
            table.append(self.free.pop())
        return True
```

Basis: Block size is the vLLM default (source-confirmed); the allocator is an illustrative sketch and the corrected rule was checked by hand, not run against a model server.

07 Choose the block size

DESIGN CHOICE

SCENARIO

Someone proposes 256-token blocks because they make the attention kernel more efficient. Another proposes 4-token blocks to waste almost nothing. The default is 16. Requests average 2,500 tokens, with many short ones in the mix.

HOW TO THINK ABOUT IT

Put a number on the cost of each extreme in the same units, then ask whose workload would move the answer.

Q What does the block size trade?

A Wasted space against per-block overhead. Small blocks waste less in each request's last block, but a request needs more blocks, a longer block table and more separate memory reads. Large blocks do the opposite.

- The paper notes a larger block uses the GPU better per kernel call and also increases fragmentation, and it chose 16 as the default after an ablation (source-confirmed).

Q How much space does each choice waste?

A On average a request's last block is half empty, so the waste is about (B − 1) ÷ 2 tokens, at 0.8 MB per token.

- **B = 16:** 7.5 tokens = 6 MB a request; 25 requests = 150 MB, about 0.3% of the 52 GB budget.
- **B = 256:** 127.5 tokens = 102 MB a request; 25 requests = 2.55 GB, about 4.9% of the budget.

Q Whose workload changes the answer?

A Short requests. A 50-token request in a 256-token block leaves 206 tokens, 80% of the block, unused.

- For long conversations of about 2,500 tokens the difference between sizes is small in relative terms. For a mix with many short requests, small blocks matter more.
- A block table for 2,500 tokens is 157 entries at B = 16 and 10 at B = 256: tiny either way.

Q What would you measure?

A Throughput and p99 latency under a replay of real traffic at two or three block sizes, with the production kernel.

- The arithmetic above bounds the memory side. Only a measurement shows the kernel side.

Q Reframe it.

A A tunable constant is a trade-off with two sides. Price both sides in the same unit, then find the workload feature that moves the balance.

Basis: Default block size and the trade-off direction are from the paper (source-confirmed); the 2,500-token average and the 50-token request are illustrative; waste figures are arithmetic (recomputed).

08 Twenty users share a system prompt

SHARING

SCENARIO

An assistant product prepends the same 1,000-token system prompt to every conversation. Twenty conversations are running on one GPU. An engineer asks whether the prompt's KV cache can be stored once instead of twenty times.

HOW TO THINK ABOUT IT

Count the memory with and without sharing, then ask how a shared block stays correct when one owner writes, and what would stop two prompts from sharing at all.

Q How much memory does sharing save?

A Without sharing: 20 × 1,000 tokens × 0.8 MB = 16,000 MB, 16 GB.

- With 16-token blocks, 62 full blocks (992 tokens, 794 MB) are shared by all twenty. Each request keeps its own last partial block (at most 12.8 MB each, 256 MB in all). Total about 1.05 GB.
- Saving: about **15 GB**, 29% of a 52 GB cache budget.

Q How does a request write to a shared block safely?

A Copy-on-write. Each shared block has a reference count. A request that must modify a shared block copies it first, takes the private copy, and the count of the original drops by one.

- This is the mechanism behind forking a process in an operating system. It means a shared prefix costs one copy and only the differing tail is private.

Q What stops two prompts from sharing?

A Any difference earlier in the prompt. A block is identified by its own tokens and the tokens before it, so one changed token near the start changes every block after it.

- Put stable text first and changing text after it. A timestamp, a user name or a request id at the top of the prompt defeats sharing for the whole prompt.
- Only full blocks are shared, so the partial block at the end of a prefix is not (vLLM's documentation).

Q What does sharing do to the first-token wait?

A It skips the prompt processing for the shared part. 992 shared tokens are 2 × 13e9 × 992 = 2.6 × 10^13 operations, about 0.09 s of GPU time saved at an assumed 300 trillion a second.

- Memory and time are saved together, which is why prefix reuse appears in the first-token plan (Question 11).

Q Reframe it.

A When the same data is needed many times, store it once and handle the exceptions. Then find what makes data not the same, since that is where the saving disappears.

Basis: Block sharing, copy-on-write, full-block caching and prefix identification are from the paper and the vLLM documentation (source-confirmed; docs, 2026-10-17); the 1,000-token prompt and 20 conversations are illustrative; the rest is arithmetic (recomputed).

PART 3

Scheduling, caching and routing

What the scheduler does when memory runs out, how a long prompt hurts everyone, and where cached prefixes live.

READ THESE FIRST: CONCEPTS COVERED IN THIS PART

Waiting queue and running batch	New requests wait in a queue. A scheduler moves them into the running batch, whose members each produce one token per step. Admission is gated on free KV blocks, not on a fixed request count.
Preemption	Taking a running request out of the batch to free its memory for others, then bringing it back later. It replaces rejecting new work when memory is short.
Swapping and recomputation	Two ways to resume a preempted request. Swapping copies its KV blocks to CPU memory and back. Recomputation drops them and rebuilds them in one pass over the prompt plus the tokens generated so far.
First-come-first-served (FCFS)	A scheduling policy in which earlier arrivals are served first. In vLLM's preemption, the latest-arrived requests are preempted first, which prevents starvation of old requests.
Chunked prefill	Splitting a long prompt's processing into chunks and batching each chunk with running decode steps, so that no stream waits for a whole long prompt to finish.
Head-of-line blocking	One slow item at the front delaying everything behind it. A single long prompt processed in one step delays every running stream.
Session affinity	Routing all requests of one conversation to the same replica, so that the replica's cached KV blocks for that conversation can be reused.
Consistent hashing	A way to assign keys to servers so that adding or removing one server moves only about 1 ÷ (N + 1) of the keys, instead of nearly all of them.
Where to read	Kwon et al.: PagedAttention (vLLM paper) | vLLM documentation: optimization and tuning | vLLM documentation: automatic prefix caching

09 The GPU is out of blocks in the middle of decoding

OPERATIONS

SCENARIO

At the evening peak, KV-cache utilization hits 100%. Every running request needs one more block on the next step and the free list is empty. Right now the process throws an exception and the whole batch fails.

HOW TO THINK ABOUT IT

Treat a full pool as a normal event with a policy, not as an error. List the options, say whom you would sacrifice and why, and price the cost of getting each victim back.

Q What are the options when no block is free?

A Reject the request, make new requests wait, or preempt a running one. Never crash.

- Rejecting and queueing protect running streams but make admission worse for new users.
- Preemption frees blocks by pausing a running request and resuming it later. It keeps accepting work at the cost of a delay for the victim.

Q Whom do you preempt?

A The paper's policy is first-come-first-served: the earliest arrivals keep their memory, and the latest-arrived requests are preempted first (source-confirmed).

- It is fair and prevents starvation. A request that has waited longest is never the one pushed out again and again.
- The tempting alternatives, preempting the longest-running or the lowest-priority request, need a stated reason, because they can starve those requests.

Q Swap or recompute? Do the arithmetic.

A For a 2,000-token request, the KV cache is 1.6 GB.

- **Swap:** at an assumed 25 GB a second over PCIe, 64 ms out and 64 ms back, about 130 ms. The CPU swap space is bounded by the GPU's KV size (source-confirmed), so at most 52 GB of host memory per GPU.
- **Recompute:** prompt processing for 2,000 tokens, about 0.17 s at peak and 0.4 s at 40% of peak.
- On paper swapping is faster, but both figures leave out real overheads. vLLM's documentation currently makes recompute the default and says it has lower overhead in the V1 architecture (docs, 2026-10-17). Measure both on your stack.

Q How do you know it is happening, and what do you do about it?

A Count preemptions. The documentation says the count is exposed as a metric and warns in the logs.

- Frequent preemption means the cache is too small for the load. Remedies, in order: allow more GPU memory for the cache, lower the number of concurrent sequences or the tokens per step, add replicas, or shrink the KV footprint (Question 15).
- Alert on the preemption rate next to KV utilization and queue depth.

Q Reframe it.

A Resource exhaustion is a design input. Decide in advance what gets sacrificed and what it costs, so that the busiest minute follows a policy and not a crash.

Basis: Preemption policy, swap bound and the two resume methods are from the paper (source-confirmed); the recompute default and metrics are from vLLM's documentation (docs, 2026-10-17); the PCIe bandwidth, the 2,000-token request and compute speeds are illustrative; arithmetic recomputed.

10 One long prompt stalls every stream

DEBUGGING

SCENARIO

p99 inter-token latency spikes to about 800 ms a few times an hour, although average utilization is moderate. The logs show that each spike coincides with a request whose prompt is around 8,000 tokens. The target is 120 ms.

HOW TO THINK ABOUT IT

A rare event that hurts everyone points to a shared resource used in one big piece. Find the piece, size it with arithmetic, and pick a fix that bounds the piece instead of removing the request.

Q Why would one request hurt all the others?

A Prompt processing is a large block of work done in one step.

- 8,000 tokens × 2 × 13 billion = 2.1 × 10^14 operations, about 0.7 s at an assumed 300 trillion a second, and about 1.7 s at 40% of peak.
- If that runs as one step, every running stream waits for it. This is head-of-line blocking, and it explains why an ordinary average hides the spikes.

Q What is the fix?

A Chunked prefill: split the prompt into chunks and run each chunk in a step together with the running decodes.

- vLLM's documentation says chunked prefill is enabled by default whenever possible, that it schedules pending decodes before prefill work, and that a prefill that does not fit into the per-step token budget is chunked automatically (docs, 2026-10-17).
- So the question is not whether to chunk but how large the chunk budget should be.

Q How do you choose the chunk size?

A From the latency budget. Decide how much prefill work a step may add, then convert it to tokens.

- To add at most about 60 ms of prefill work to a step: 0.06 s × 300 trillion ÷ (2 × 13 billion) = about **690 tokens** at peak, about **280** at 40% of peak.
- A 2,048-token chunk is about 0.18 s of work at peak, already more than the 120 ms p99 target, even though it is a common published example. Tune on your hardware.
- The documentation says smaller budgets give better inter-token latency, and budgets above 8,192 are recommended for throughput. The cost of small chunks is a longer first-token time for the long prompt itself.

Q How do you confirm the diagnosis?

A Slice the data.

- Plot inter-token p99 by the length of the longest prompt in each step's batch. A diagnosis is right if the spikes follow prompt length.
- After the change, compare p99 inter-token latency and the long prompt's own first-token time. Expect one to fall and the other to rise.

Q What other options exist?

A Cap prompt length, or send long prompts to a separate pool so they cannot share steps with interactive streams.

- Both trade complexity or product limits for isolation. Choose them only if chunking cannot meet the target.

Q Reframe it.

A When a rare event hurts everyone, bound its size. A shared resource used in one big piece will produce a tail no average can show.

Basis: Chunked-prefill behavior and token-budget guidance are from vLLM's documentation (docs, 2026-10-17); the 8,000-token prompt, 800 ms spike and 300 trillion operations a second are illustrative; the chunk-size arithmetic is recomputed.

11 Keep the conversation's cache, or recompute it?

CACHE DESIGN

SCENARIO

Users in a chat product reply about 60 seconds after each answer. The context is 2,000 tokens. The team wants first-token times near 200 ms and asks whether to keep each conversation's KV cache on the GPU between turns.

HOW TO THINK ABOUT IT

Price both ways of getting the cache back in the same unit, GPU-seconds, then ask what a hit rate must be to meet the target, and what the data to decide it with is.

Q What does a cache hit save?

A The prompt processing for the cached part.

- Cold: about 0.17 s at peak and 0.4 s at 40% of peak for 2,000 tokens. Cached: 100 new tokens cost about 9 ms.
- So a hit saves roughly 0.16 to 0.42 s of GPU time per turn and moves the first-token time from about 0.4 s to about 0.01 s.

Q What does it cost to park the cache on the GPU while the user thinks?

A The cache holds 2.0 GB, one of the GPU's 25 request slots. For 60 seconds that slot is worth 60 ÷ 25 = **2.4 GPU-seconds** of capacity.

- Recomputing costs 0.17 to 0.43 GPU-seconds. Parking on the GPU is roughly 6 to 14 times more expensive than recomputing.
- So idle caches should not be reserved on the GPU.

Q So where should an idle cache live?

A In places that are free when memory is not needed.

- vLLM keeps hashed blocks of finished requests in the free pool and evicts the least recently used when memory is needed (docs, 2026-10-17). A hit costs nothing extra, and the memory is never reserved.
- A larger, slower tier makes the hit last longer: host memory holds about 2 GB per conversation, so an assumed 512 GB of host memory holds about 256 idle conversations per machine.
- Otherwise accept the recompute.

Q What hit rate does the target need?

A p50 is the median, so more than half of requests must be fast. With a cold cost of about 0.4 s and a hit cost of about 0.01 s, a p50 under 200 ms needs a hit rate above 50%, and above about 60% to leave a margin.

- Track the hit rate as a first-class metric, by conversation age and by tenant. If it is 20%, the 200 ms target is unreachable however fast the hardware.

Q What did Character.AI say about this?

A Its post credits improved techniques for inter-turn caching among several changes, and gives no hit rate (source-confirmed).

- Use it as a hint about which mechanism matters, not as a number. Measure your own hit rate on your own conversation gaps.

Q Reframe it.

A Caching is a trade between holding a result and recomputing it. Put both costs in one unit, then find the break-even.

Basis: Block retention and LRU eviction are from vLLM's documentation (docs, 2026-10-17); Character.AI's description is from its post (source-confirmed); the 60-second gap, the 512 GB host memory and the compute speed are illustrative; arithmetic recomputed.

12 The load balancer is round-robin

ROUTING

SCENARIO

Eight replicas sit behind a round-robin load balancer. The prefix-cache hit rate is 12%, first-token times are poor, and chat users send several turns per conversation.

HOW TO THINK ABOUT IT

Ask where the cached state physically lives and what the routing rule does to it. Then price the fix by what it breaks.

Q Why is the hit rate near 12%?

A A conversation's cache sits on the replica that served its last turn. Round-robin sends the next turn to a replica chosen without regard to that, so the chance of landing on the right one is 1 in 8 = 12.5%.

- The observed rate matches the arithmetic, which is a strong sign the cause is routing and not cache size.

Q What is the fix?

A Session affinity: route by conversation id or by a hash of the prompt prefix, so a conversation keeps going to the same replica (Lesson 4, Concept 3).

- With affinity the hit rate is limited by cache eviction and conversation gaps, not by routing.

Q What does affinity cost?

A Load balance, and a little availability.

- A heavy conversation or a popular prefix pins load to one replica. Allow spillover to another replica when the preferred queue is long, accepting a cold prompt in exchange.
- If a replica dies, its conversations lose their caches. That is only a recompute, not lost data.

Q What happens when you add a replica?

A It depends on the hashing. With a plain hash modulo N, going from 8 to 9 replicas remaps about 8 ÷ 9 = **89%** of conversations, and nearly every cache is cold at once.

- With consistent hashing only about 1 ÷ 9 = **11%** move.
- Plan scale-out for quiet hours or one replica at a time.

Q How do you verify the change?

A Per-replica hit rate, p50 first-token time before and after, and load skew (busiest replica over the mean).

- A good change raises the hit rate sharply while keeping skew bounded.

Q Reframe it.

A State lives somewhere. A routing rule that ignores where it lives throws it away, so design routing and caching together.

Basis: Round-robin and hashing arithmetic are exact; the eight replicas and the 12% hit rate are illustrative; session affinity is from Lesson 4, Concept 3.

PART 4

Operating at scale: dashboards, availability, growth and safety

Reading the signals, promising an availability number, planning for ten times the load and closing the side channels.

READ THESE FIRST: CONCEPTS COVERED IN THIS PART

KV-cache utilization	Blocks in use divided by total blocks. It is the most direct health signal because memory is the binding resource.
Queue depth	The number of requests waiting for a batch slot. Growth at high utilization means real capacity exhaustion, not a blip.
Availability and downtime	Availability is the share of time a service works. 99.9% is 8.76 hours of downtime a year. Components in series multiply their availabilities, and a redundant pair fails only when both parts fail.
Back pressure	When work arrives faster than it drains, bound the queue and slow or shed load, instead of letting delay grow without limit.
Side channel	Information leaking through something other than the intended output, such as timing. A fast first token can reveal that a prefix was already cached.
Cache salting	Adding a secret value to the cache key, so only requests with the same salt can reuse cached blocks. It isolates tenants.
Noisy neighbor	One tenant's heavy use degrading service for others who share the same resource.
Cancellation propagation	When a client disconnects, the server stops generating and frees the request's blocks immediately, instead of producing tokens nobody will read.
Where to read	Kwon et al.: PagedAttention (vLLM paper) | vLLM documentation: automatic prefix caching | Character.AI: Optimizing AI Inference

13 Three incidents, one dashboard

MONITORING

SCENARIO

In one week the dashboard shows three different problems. (A) KV utilization 99%, queue depth growing, p99 first-token time rising. (B) p99 first-token time breached while utilization is 55% and the queue is short. (C) Cost per million tokens up 30% with flat latency and flat traffic.

HOW TO THINK ABOUT IT

The same symptom, a slow service, has different causes that need different remedies. Read each signal as a question: is the system out of capacity, misconfigured or under-used?

Q What is A, and what do you do?

A A real capacity signal. Memory is full and demand exceeds what the cache can hold.

- Check the preemption count: high means requests are being paused and resumed, which adds delay.
- Remedies: add replicas, or shrink the KV footprint (Question 15), or apply back pressure.

Q What is B?

A Not a capacity problem: more hardware will not fix a breach at 55% utilization.

- Look for stalls and misses: a long prompt blocking decodes (Question 10), a falling prefix-cache hit rate after a routing change (Question 12), or a preemption storm.
- Slice p99 first-token time by prompt length, by replica and by cache hit or miss to find which.

Q What is C?

A Fewer tokens served for the same GPU spend. +30% cost per token with a flat fleet means the fleet served 1 ÷ 1.3 = 77% of the tokens, a 23% drop.

- Candidates: utilization fell (idle capacity after a scale change), the mix shifted to long contexts so fewer requests fit per GPU, or the hit rate fell so more prompt work is recomputed.
- Cost by tenant and by prompt length shows which.

Q What do you put on the dashboard so these are distinguishable?

A KV utilization, queue depth, preemption count, p50 and p99 for both latencies, cache hit rate, cost per million tokens and utilization of GPUs.

- Each signal answers one question. Together they separate capacity, configuration and efficiency.

Q Reframe it.

A A symptom has many causes. Instrument so that each cause moves a different signal.

Basis: Signal-to-remedy mapping follows Step 6 of the case study (reasoned); the percentages are illustrative; the 77% and 23% are arithmetic (recomputed).

14 What availability can we promise?

AVAILABILITY

SCENARIO

A gateway, an inference backend and a vector database sit in series. Their availabilities are 99.95%, 99.9% and 99.99%. Sales wants to promise 99.99% for the whole chain.

HOW TO THINK ABOUT IT

Multiply, find the weakest part, and add redundancy where it pays most. Then ask what the formula leaves out before you quote a number.

Q What is the availability of the chain?

A The product of the three: 0.9995 × 0.999 × 0.9999 = 0.998401, about 99.84%, which is about **14 hours** of downtime a year.

- A chain is worse than its worst part, because any failure breaks it.

Q Where does redundancy pay most?

A At the weakest part, the inference backend (99.9%, 8.76 hours a year alone).

- Two independent replicas fail together with probability $0.001^2 = 0.000001$, so the pair is 99.9999% available.
- The chain becomes 0.9995 × 0.999999 × 0.9999 = 0.999399, about 99.94%, or about **5.3 hours** a year.

Q Can you reach 99.99%?

A Not with one change. Pair the gateway as well: $1 - 0.0005^2 = 0.99999975$, and the chain is about 99.99%, 53 minutes a year.

- Now the vector database (99.99%, 53 minutes alone) is the weakest part. Each step costs another set of machines.

Q What does the formula leave out?

A It assumes independent failures, and it counts a part as simply up or down.

- Replicas share failure causes: the same driver bug, the same bad deploy or the same rack. Real redundancy gains are smaller.
- Failover does not save in-flight streams, and the replica that takes over has a cold cache, so first-token times spike. Decide whether a slow service counts as down in the SLA.

Q What do you promise?

A Less than the arithmetic. For example 99.9%, 8.76 hours a year, with the computed 99.94% as headroom.

- Promise what the weakest assumption supports, not what the formula gives.

Q Reframe it.

A A computed number is an upper bound. Quote a number below it, and say which assumptions the bound rests on.

Basis: Component availabilities and the series and parallel formulas are from Lesson 3, Concept 3 (illustrative values); all downtime figures are arithmetic (recomputed).

15 Traffic grows tenfold

SCALE

SCENARIO

The product succeeds. Concurrent conversations are expected to grow from 1,000 to 10,000 in a year. Your manager asks what it costs and what breaks first.

HOW TO THINK ABOUT IT

Scale each quantity by ten and see which grows faster than the budget. Then list the levers by what each one costs in quality, effort and risk.

Q What does ten times do to the plain plan?

A Everything in the fleet scales with concurrency: 10,000 ÷ 25 = **400 GPUs**, $1,000 an hour at $2.50, still about $1.11 per million tokens at full utilization.

- What breaks first is not the allocator. It is capacity planning: GPU supply, the time to add machines, a single gateway tier and the utilization you can sustain through the daily cycle.

Q Which levers shrink the fleet?

A Memory levers, each with a quality price.

- 8-bit weights: 13 GB for weights, 65 GB for cache, 32 requests per GPU, 10,000 ÷ 32 = **313 GPUs**.
- 8-bit weights and an 8-bit KV cache: 1.0 GB per request, 65 per GPU, **154 GPUs**.
- A model with grouped-query attention would shrink the cache further, but that is a model choice, not a serving setting.
- Each lever needs an evaluation on your tasks before it ships (Lesson 9).

Q What about demand spikes?

A A burst uses up headroom first and then queues. p99 first-token time is the first thing to break.

- Apply back pressure: bound the queue, then degrade in order of least harm: cap output length, shorten context, route a share of traffic to a smaller model through a gateway (capstone #15), and only then reject.
- Autoscaling takes minutes to bring up a replica, so it does not help with a spike of seconds.

Q What changes in operations at 400 GPUs?

A Failures become routine, and blast radius matters.

- With 400 GPUs, some will fail every week. Pool replicas into failure domains small enough that one failure costs a small share of capacity.
- Rolling deploys, per-pool canaries and per-tenant limits become necessary, not nice to have.

Q Reframe it.

A Scaling is not one number. Find the term that grows faster than your budget or your control, and plan for that one.

Basis: GPU counts are arithmetic from Question 2's sizing (recomputed); the GPU price is illustrative; the quality effect of 8-bit weights or cache is not asserted and must be measured.

16 Two tenants share a prefix cache

SECURITY

SCENARIO

The API is multi-tenant. Automatic prefix caching is on because it made first-token times much better. A security reviewer asks what could go wrong.

HOW TO THINK ABOUT IT

Whenever two parties share something that makes a request faster, ask what the speed reveals. Then weigh isolation against what it costs in memory and speed.

Q What is the risk?

A A timing side channel. A request whose prefix is already cached gets a much faster first token, so a caller can guess a prefix and learn from the speed whether another tenant already sent it.

- That can reveal a competitor's system prompt, a document in someone's prompt or a template. The reviewer is right to ask.

Q What does the documentation offer?

A Cache salting: a secret value added to each block's identity, so only requests with the same salt can reuse cached blocks (docs, 2026-10-17).

- The documentation also notes that blocks are identified by a hash, that the default hashing is collision-resistant in recent versions, and that non-cryptographic hash options carry collision risks in multi-tenant settings.

Q What does isolation cost?

A Sharing across tenants.

- If 20 tenants each send the same public 1,000-token system prompt, isolated caches hold 20 copies, 20 × 0.8 GB = 16 GB, against 0.8 GB shared. That is up to 15 GB, 29% of a 52 GB cache budget, and 20 cold prompt computations instead of one.
- A reasonable policy: share prefixes that the operator itself supplies and that are public. Salt per tenant for anything the tenant supplies.

Q What other abuse cases belong in the review?

A Noisy neighbors, cancellation and logging.

- One caller with very long contexts can take most of the block pool: cap context length and in-flight requests per tenant and meter tokens (Lesson 10).
- When a client disconnects, stop generating and free the blocks, or capacity goes to tokens nobody reads.
- Logging full prompts stores sensitive text. Apply the same retention and access rules as any user data (Step 3 of the case study).

Q Reframe it.

A Every optimization that shares state across parties is a possible leak. Decide who may share with whom, and pay the cost of the isolation you need.

Basis: Cache salting, hashing and the shared-block behavior are from vLLM's documentation (docs, 2026-10-17); the side-channel risk follows from that design (reasoned); the 20 tenants and the 1,000-token prompt are illustrative; arithmetic recomputed.

17 A blog post says 33 times cheaper

EVIDENCE

SCENARIO

An executive forwards a post from Character.AI: “reduced our serving costs by at least 33X.” They ask the team to “do what they did” and promise the same saving next year.

HOW TO THINK ABOUT IT

Read what the claim says, against what, and made of what. Then separate the mechanism you can borrow from the number you cannot.

Q What does the post actually say?

A It states that since launching in 2022 the company reduced serving costs by at least 33 times, and that serving costs 13.5 times less than building on the most efficient leading commercial APIs.

- It credits innovations around the Transformer architecture and the attention KV cache, and improved techniques for inter-turn caching.
- It says the company serves around 20,000 queries a second at under a cent per hour of conversation. It names no author and gives no cache hit rate (source-confirmed, blog.character.ai, June 20, 2024).

Q What is the 33 times measured against?

A The company's own 2022 costs, and the post does not say what that baseline was.

- It is a bundle of changes over about two years, not the gain from one technique.
- The architectural part is a model-design decision made when the models are trained. It cannot be applied to a model you only serve.

Q What can you borrow?

A The mechanisms, not the number.

- Block-level memory management (Part 2), prefix and inter-turn reuse (Questions 11 and 12) and routing that preserves cached state.
- Each has a measurable effect on your hardware and traffic: measure your own hit rate and your own memory waste.

Q What do you tell the executive?

A Show your own estimate with its sources and assumptions.

- For this system: block allocation is worth about three to five times the fleet against contiguous allocation (Question 2); 8-bit weights could cut the fleet by about 20% (Question 4), subject to quality tests; caching depends on a hit rate you have not measured.
- Offer a range with the measurement that would narrow it, not a promise copied from someone else's baseline.

Q Reframe it.

A A headline ratio has a baseline and a bundle. Ask what it is measured against and what it is made of, then borrow the mechanisms and re-derive the number.

Basis: Quotations and figures are from the Character.AI post (source-confirmed); the system estimates are arithmetic from earlier questions (recomputed); the share of 8-bit savings is illustrative until tested.

Sources and confidence

The case study this book is built on is the AI System Design Primer's Capstone #10. This revision read the full text of the vLLM paper, the Character.AI post and two vLLM documentation pages on 2026-10-17. Nothing described here was run on real systems: the allocator code is a sketch, and the bandwidth, compute and price figures are round illustrative numbers. Scenario figures are illustrative.

- Kwon et al.: Efficient Memory Management for LLM Serving with PagedAttention, SOSP 2023 (arXiv 2309.06180), full text read 2026-10-17
https://arxiv.org/abs/2309.06180
- Character.AI: Optimizing AI Inference at Character.AI (June 20, 2024), read 2026-10-17
https://blog.character.ai/optimizing-ai-inference-at-character-ai/
- vLLM documentation: optimization and tuning (preemption, chunked prefill, token budgets), read 2026-10-17
https://docs.vllm.ai/en/latest/configuration/optimization/
- vLLM documentation: automatic prefix caching (block hashing, eviction, cache salting), read 2026-10-17
https://docs.vllm.ai/en/latest/design/prefix_caching/